

PRESS RECORD

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Solution Quality

COMPX₃₄₁

Before we start ...

- Testing Standard Out
- Functional Software Test Plan Example

Testing Standard Out

Testing Standard Out: setup

```
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.Assertions.*;
import java.io.*;

public class TestClass {

    private final PrintStream standardOut = System.out;
    private final ByteArrayOutputStream outputStreamCaptor = new ByteArrayOutputStream();

    @BeforeEach
    public void setOut() {
        System.setOut(new PrintStream(outputStreamCaptor));
    }

    // Add tests here

}
```

Testing Standard Out: tests

```
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.Assertions.*;
import java.io.*;

public class TestClass {
    ...
    // Add tests here
    @Test
    @DisplayName("Test Print Method")
    public void testPrint() {
        SomeClass sc = new SomeClass();
        sc.printSomething();
        String actual = outputStreamCaptor.toString().trim();
        Assertions.assertEquals("Some text", actual);
    }
}
```

Testing Standard Out

- Let's give it a try ...

Functional Software Test Plan Example

Functional Software Test Plan Example

- Let's take a look ...

Solution Quality

COMPX₃₄₁

Overview

- Solution quality – why does it matter?
- Minimizing Complexity
- Anticipating Change
- Construction for Verification
- Reuse
- Standards in Construction

Solution Quality – why does it matter?

Solution quality isn't about bugs, it's about structuring your software well. Badly structured software may slow down development or increase the risk of introducing bugs.

Remember from the design content:

- Decomposition/modularisation

- Encapsulation/information hiding

- Sufficiency, completeness, and primitiveness

- Separation of concerns

- Separation of interface and implementation

Solution Quality - fundamentals

1. Minimizing Complexity
2. Anticipating Change
3. Construction for Verification
4. Reuse
5. Standards in Construction

1. Minimizing complexity

- Code creation that is simple and readable, rather than clever
- We can do this by making use of:
 - Standards in Construction
 - Modular Design
 - Coding techniques
 - Code quality

Minimizing complexity (Example)

"We would like to create a spell checker application. The application will take a word and either confirm that it is spelt correctly, or confirm that it is not spelt correctly. Input words may include punctuation, but cannot include numerical characters"

Minimizing complexity (Example)

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word.toLowerCase();
wordCleaned = wordCleaned.replaceAll("[\\p{Punct}\\p{IsPunctuation}]", "");

// Check if word contains a number
Boolean hasNumerical = false;
for(int i = 0; i < wordCleaned.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(wordCleaned.charAt(i)))){
        hasNumerical = true;
    }
}

// Check if it is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = false;
    for(int i = 0; i < dictionary.length; i++){
        if(dictionary[i].equals(wordCleaned)){
            isWord = true;
        }
    }

    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word;
wordCleaned = convertToLower(wordCleaned);
wordCleaned = stripPunctuation(wordCleaned);
Boolean hasNumerical = includesNumerical(wordCleaned);

// Determining whether the word is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = existsInDictionary(wordCleaned);
    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

Minimizing complexity (Reading)

- <https://www.techtarget.com/searchsoftwarequality/tip/How-to-prevent-accidental-complexity-in-software-development>

2. Anticipating change

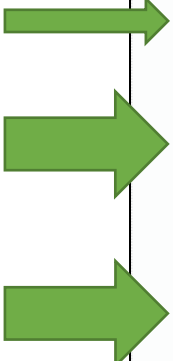
- Most Software will change over time
 - More features added
 - Maintenance performed
 - Change of environment
 - ...

Anticipating change (Example)

"We would like to create a spell checker application. The application will take a word and either confirm that it is spelt correctly, or confirm that it is not spelt correctly. Input words may include punctuation, but cannot include numerical characters"

" added feature, the spell checker should be able to accept words that include numerical characters – these characters will just be stripped from the word"*

Anticipating change (Example)



```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word.toLowerCase();
wordCleaned = wordCleaned.replaceAll("[\\p{Punct}\\p{IsPunctuation}]", "");

// Check if word contains a number
Boolean hasNumerical = false;
for(int i = 0; i < wordCleaned.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(wordCleaned.charAt(i)))){
        hasNumerical = true;
    }
}

// Check if it is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = false;
    for(int i = 0; i < dictionary.length; i++){
        if(dictionary[i].equals(wordCleaned)){
            isWord = true;
        }
    }

    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

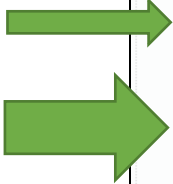
// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word.toLowerCase();
wordCleaned = wordCleaned.replaceAll("[\\p{Punct}\\p{IsPunctuation}]", "");
wordCleaned = wordCleaned.replaceAll("[0-9]", "");

Boolean isWord = false;
for(int i = 0; i < dictionary.length; i++){
    if(dictionary[i].equals(wordCleaned)){
        isWord = true;
    }
}

if(isWord){
    System.out.println("It's a word!");
} else {
    System.out.println("It's NOT a word!");
}
```

Anticipating change (Example)



```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word;
wordCleaned = convertToLower(wordCleaned);
wordCleaned = stripPunctuation(wordCleaned);
Boolean hasNumerical = includesNumerical(wordCleaned);

// Determining whether the word is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = existsInDictionary(wordCleaned);
    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word;
wordCleaned = convertToLower(wordCleaned);
wordCleaned = stripPunctuation(wordCleaned);
wordCleaned = stripNumerals(wordCleaned);

// Determining whether the word is a 'real' word
Boolean isWord = existsInDictionary(wordCleaned);
if(isWord){
    System.out.println("It's a word!");
} else {
    System.out.println("It's NOT a word!");
}
```

3. Constructing for Verification

- When you are building the software, think about how it will be tested.
- We can do this by making use of:
 - Coding standards
 - Code reviews and unit testing
 - Organizing code to support automated testing
 - Restricting the use of complex or hard-to-understand language structures
 - ...

Constructing for Verification (Example)

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word.toLowerCase();
wordCleaned = wordCleaned.replaceAll("[\\p{Punct}\\p{IsPunctuation}]", "");

// Check if word contains a number
Boolean hasNumerical = false;
for(int i = 0; i < wordCleaned.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(wordCleaned.charAt(i)))){
        hasNumerical = true;
    }
}

// Check if it is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = false;
    for(int i = 0; i < dictionary.length; i++){
        if(dictionary[i].equals(wordCleaned)){
            isWord = true;
        }
    }

    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

- Unit tests?
- Main method

Constructing for Verification (Example)

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word;
wordCleaned = convertToLower(wordCleaned);
wordCleaned = stripPunctuation(wordCleaned);
Boolean hasNumerical = includesNumerical(wordCleaned);

// Determining whether the word is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = existsInDictionary(wordCleaned);
    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

- Unit tests?
 - convertToLower()
 - stripPunctuation()
 - includesNumerical()
 - existsInDictionary()
 - Main method

4. Reuse

- Using existing assets to solve different problems
- Assets include:
 - Libraries
 - Modules
 - Components
 - Source code
 - Commercial off-the-shelf assets
- Note “construction *for* reuse” versus “construction *with* reuse”

Reuse (Example)

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word.toLowerCase();
wordCleaned = wordCleaned.replaceAll("[\\p{Punct}\\p{IsPunctuation}]", "");

// Check if word contains a number
Boolean hasNumerical = false;
for(int i = 0; i < wordCleaned.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(wordCleaned.charAt(i)))){
        hasNumerical = true;
    }
}

// Check if it is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = false;
    for(int i = 0; i < dictionary.length; i++){
        if(dictionary[i].equals(wordCleaned)){
            isWord = true;
        }
    }

    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

- What can we reuse?
 - Overall method

Reuse (Example)

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word;
wordCleaned = convertToLower(wordCleaned);
wordCleaned = stripPunctuation(wordCleaned);
Boolean hasNumerical = includesNumerical(wordCleaned);

// Determining whether the word is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = existsInDictionary(wordCleaned);
    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

- What can we reuse?
 - convertToLower()
 - stripPunctuation()
 - includesNumerical()
 - existsInDictionary()
 - Overall method

5. Standards in Construction

Standards can include:

- Communication methods (e.g. document formats and contents)
- Programming languages (e.g. language standards)
- Coding standards (e.g. naming conventions etc.) – more about this next time!
- Platforms (e.g. interface standards for system calls)
- Tools (e.g. diagrammatic standards like UML)

Also, note, external versus internal standards.

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato